

Table des matières

Reprise de la branche <code>apij c</code> dans la ligne 2.x	1
1. Pourquoi la branche n'a pas pu être fusionnée	1
2. Objectifs du cahier des charges — état d'intégration	1
3. Détail des six commits de reprise	2
4. Ce qui n'a pas été repris, et pourquoi	3
5. Deux différences qui casseront tes scripts existants	4
6. Questions de schéma encore ouvertes	4
7. Ce qui est plus riche en 2.x qu'en <code>apij c</code>	5
8. Décisions prises (J-C. Passieux, 16 septembre)	5

Reprise de la branche `apij c` dans la ligne 2.x

Document de travail pour la discussion avec Jean-Charles Passieux. État au 16 septembre 2026. **Décisions de J-C. Passieux intégrées** (section 8).

1. Pourquoi la branche n'a pas pu être fusionnée

`apij c` contient **un seul commit** (94675f8, 7 septembre 2026), qui touche 27 fichiers (23 nouveaux, 4 modifiés) pour environ 18 000 lignes ajoutées.

Sa base commune avec `develop` est 78ce687 : c'est-à-dire l'**ancien main, ligne 1.x, schéma 2024.7.1**. La refonte 2.x (schéma 2026.9.8) s'est faite en parallèle sur `develop`. Les deux lignes ont divergé le 3 avril 2026.

Fusionner la branche aurait donc réintroduit l'ancien schéma et l'ancienne architecture. Elle a été traitée comme un **cahier des charges**, pas comme du code à intégrer.

Le travail est préservé par le tag **`apij c-snapshot`**, poussé sur le dépôt. Il pointe sur 94675f8 indépendamment de la branche : celle-ci peut être supprimée sans rien perdre.

2. Objectifs du cahier des charges — état d'intégration

2.1 Demandes explicites (section « Beta-test de R3XA_API > évolutions »)

Demande	État	Où
Classe par type, validable / printable / save / load indépendamment du fichier R3XA	✓ Intégré	R3XAItem (chemin dict) et <code>r3xa_api.models.*</code> (chemin typé)
Méthode pour lister les attributs optionnels	✓ Intégré, et élargi	<code>optional_fields()</code> , <code>required_fields()</code> , <code>missing_fields()</code> , <code>field_descriptions()</code>
Corriger le <code>time_reference</code> de <code>dataset_file</code>	✓ Intégré	<code>\$ref: types/unit</code> dans le schéma 2026.9.8
Installation simplifiée <code>pip install r3xa</code>	⊗ Non traité	Assigné à Manu dans le compte-rendu

2.2 Le bloc de réalisations décrit dans le compte-rendu

Ce que la branche apportait, et ce qu'il en est aujourd'hui :

Fonctionnalité de apijc	État	Comment
Génération des classes depuis <code>schema.json</code>	✓ Équivalent	<code>datamodel-code-generator</code> (outil standard) remplace <code>codegen_*.py</code> (~4 400 lignes maison)
Classe parente + classes spécialisées	✓ Équivalent	R3XAModel + 21 classes générées
Tous les attributs du schéma, initialisés à None	✓	
id optionnel, généré par <code>_random_id()</code>	✓	R3XAModel.__init__
init imposant les requis, <code>to_dict()</code> , <code>validate()</code> , <code>save()</code> , <code>load()</code> , <code>print()</code> , <code>from_dict()</code>	✓	Les 6 méthodes présentes sur les modèles et sur R3XAItem
Compatibilité registry	✓	Registry, RegistryItem
Documentation auto dans la docstring de classe	✓ Restauré	commit 9c472d1 — 21/21 classes
Documentation par attribut : type, description, REQUIRED	✓ Restauré	idem, + valeurs autorisées pour les énumérations
<code>print()</code> affichant tous les attributs, même à None	✓	
Requis marqués par *	✓	
R3XA.UNIT affichés 1392 px	✓ Restauré	commit 9c472d1
Listes d'unités [1392 px, 1040 px]	✓ Restauré	idem

2.3 Points relevés à la relecture du 16 septembre

Constat	Vérifié	Traité
« Le <code>print()</code> au niveau document a été ajouté »	✓	83b2414
« Le plot matplotlib est reparti sur d'autres couleurs »	✓ Exact	dd879e9 — palette en option
« Je ne trouve pas les classes <code>data_sources / data_sets</code> »	✓ Exact	cd2a21e — elles n'étaient exposées que sous <code>r3xa_api.models.*</code>
« <code>add_camera_source</code> fait des dictionnaires, pas des objets »	✓ Exact	f4c2b26 — R3XAItem
« J'avais ajouté <code>__str__ / __repr__</code> pour <code>print(my_file)</code> »	✓ Exact	cd2a21e

3. Détail des six commits de reprise

Commit	Objet
9c472d1	Formatage lisible de <code>print()</code> + docstrings générées depuis le schéma
83b2414	<code>R3XAFile.summary() / print() / plot()</code>
cd2a21e	Classes exportées au premier niveau + <code>print(document)</code> et <code>print(item)</code>
dd879e9	Palette document sur les trois moteurs de graphe
f4c2b26	R3XAItem : les <code>add_*</code> renvoient des objets
90e54b1	Documentation, CHANGELOG, scripts d'exemple, artefact de registre

Tous validés par la CI sur Python 3.9 à 3.13.

3.1 Le point central : R3XAIItem

Contrainte. R3XAFFile doit fonctionner sans pydantic, qui est un extra optionnel (`pip install r3xa-api[typed]`). Il n'était donc pas possible d'y renvoyer des modèles typés.

Solution. L'ergonomie a été placée sur le dictionnaire lui-même. R3XAIItem est un vrai sous-type de `dict: item["title"]`, `isinstance(item, dict)`, `json.dumps(item)`, `{**item}` et l'égalité avec un dict simple se comportent exactement comme avant.

Preuve de non-régression : **les 167 tests existants sont passés sans la moindre retouche.**

```
mon_specimen = my_file.add_specimen_setting(
    title="Openhole sample",
    description="Glass-epoxy specimen",
    sizes=[unit(title="width", value=30.0, unit="mm", scale=1.0)],
)
mon_specimen.print()

settings/specimen
* id: apcfundcawahublaaxlipnoi
* kind: settings/specimen
* title: Openhole sample
  description: Glass-epoxy specimen
  cad: None
  sizes: [30 mm]
  patterning_technique: None
  patterning_feature_size: None
```

`my_file.settings` est une liste de ces objets. `add_*` renvoie l'élément **stocké** : le modifier met à jour le document.

3.2 La palette

Ses couleurs : settings #c4894f (ocre), data_sources #bf0040 (carmin), data_sets #038181 (sarcelle). Ajoutées **en option**, pour ne pas restyler les artefacts déjà commités :

```
f.plot("graphe", palette="document") # graphviz, SVG
f.plot("graphe", backend="pyvis", palette="document") # HTML interactif
f.plot("graphe", backend="matplotlib", palette="document") # PNG
```

Une adaptation à valider. Ses boîtes étaient des aplats pleins avec texte blanc. La couleur du texte n'est pas pilotable uniformément par les trois moteurs — le backend `networkx` n'honore que `shape`, `penwidth`, `color` et `fillcolor`. Du texte noir sur #bf0040 serait illisible. Sa teinte a donc été placée **sur la bordure**, avec un fond en version claire, ce qui est la convention que suit déjà la palette par défaut. Les formes sont inchangées.

Rendus disponibles pour juger sur pièce, sur le document Qi Hu : `examples/artifacts/graph_qi_document.svg`, `graph_qi_document_nx.png`, `graph_qi_document_pyvis.html`.

4. Ce qui n'a pas été repris, et pourquoi

Élément	Décision	Motif
<code>codegen_datasets/datasources/settings.py</code> (~4 400 l.)	Repris	<code>datamodel-code-generator</code> fait la même chose, sans dette de maintenance
<code>new_dataset/new_datasource/new_settings.py</code> (~4 600 l.)	Repris	Ce sont les fichiers <i>générés</i> par les précédents
<code>schemaplot.py</code> (850 l.)	Abandonné (décision JC)	« Le rendu actuel est ok » : les trois moteurs (<code>graphviz</code> , <code>pyvis</code> , <code>networkx</code>) suffisent.
<code>schemaplot2.py</code> (528 l.) + <code>r3xa_flowchart.png</code>	Abandonné (décision JC)	Idem.
4 notebooks <code>basic_jcp_*.ipynb</code>	Non portés (décision JC)	« On va nous-même tout réécrire ».

Élément	Décision	Motif
essai-torsion.py, essai-stereorig.py, essai_tous_types.py devices/*.json (2 fiches matériel)	Non portés (décision JC)	Idem : réécriture par l'équipe.
save/*.json (5 documents)	Abandonnés	Leur place serait dans R3XA_REGISTRY si l'équipe les réécrit Sorties régénérables. Vérifié : 0 sur 3 valident sous le schéma actuel.

5. Deux différences qui casseront tes scripts existants

5.1 Noms de classes — 14 sur 18 ont changé

Règle : *Settings → *Setting (singulier), *DataSource → *Source. GenericSetting était déjà au singulier dans apijc : inchangé. Les trois *DataSet aussi.

Décidé (point 6) : des alias sont désormais fournis. CameraDataSource, SpecimenSettings... restent importables et pointent sur les classes actuelles. La condition d'iso-propriétés a été vérifiée classe par classe : **12 sur 14** ont des jeux de champs strictement identiques. Les deux exceptions sont TestingMachineSettings et StereorigSettings, dont le champ associated_data_sources est devenu attached_data_sources : le nom de classe résout, mais cet argument-là doit être renommé.

apijc	2.x
TestingMachineSettings	TestingMachineSetting
SpecimenSettings	SpecimenSetting
StereorigSettings	StereorigSetting
GenericDataSource	GenericSource
CameraDataSource	CameraSource
InfraredDataSource	InfraredSource
TomographDataSource	TomographSource
LoadCellDataSource	LoadCellSource
StrainGaugeDataSource	StrainGaugeSource
PointTemperatureDataSource	PointTemperatureSource
DicMeasurementDataSource	DicMeasurementSource
MechanicalAnalysisDataSource	MechanicalAnalysisSource
IdentificationDataSource	IdentificationSource
StrainComputationDataSource	StrainComputationSource
GenericDataSet, ListDataSet, FileDataSet	inchangés

5.2 Les arguments positionnels ne passent plus

essai_tous_types.py écrit r3xa.CameraDataSource('', 1, '', '', '', ''). Pydantic impose les mots-clés : R3XAModel.__init__() takes 1 positional argument but 3 were given.

Réserve, même avec les alias. Tes scripts passent leurs arguments en positionnel : les alias font résoudre le *nom*, pas l'appel. essai_tous_types.py devra donc être repris de toute façon.

6. Questions de schéma encore ouvertes

Elles sont **cassantes**. Tant qu'on est en 2.0.0rc1, les trancher est gratuit ; après la 2.0.0 finale, chacune coûte une version majeure. C'est le point le plus urgent de cette liste.

6.1 authors — ✓ décidé : on adopte {name, affiliation, orcid}

Tu proposais un tableau d'objets {nom, affiliation, orcid}. La 2.x a livré un tableau de chaînes avec author_orcids **en parallèle**. Ta proposition est techniquement meilleure, et c'est démontrable :

L'invariant « un ORCID par auteur » n'est pas exprimable en JSON Schema. Il est donc codé à la main, en Python, dans `validate.py:73`. Conséquence mesurée : un document avec **3 auteurs et 1 ORCID** est **accepté par le schéma seul** — c'est-à-dire par `ajv` dans un navigateur, par la couche MATLAB (qui ne mentionne nulle part `orcid`), ou par tout labo validant `schema.json` avec son propre outil.

La description du champ le dit elle-même : « *ORCIDs parallel to authors; use null when an author has no ORCID* ». Il faut bourrer de `null` pour tenir l'alignement.

Avec un tableau d'objets, l'invariant devient **structurel** : impossible à violer, dans n'importe quel langage, sans code de validation ad hoc. En prime, l'affiliation devient exprimable — la forme actuelle ne peut pas la porter.

6.2 `data_sources` → `produced_by`, `associated_data_sources` → `equipped_with`

Recommandation : ne pas faire. La 2.x a déjà renommé ces champs en `parent_data_sources` et `attached_data_sources` (0 occurrence des anciens noms). Un second renommage ferait migrer deux fois tous les documents et le registre dans le même cycle de version, pour un gain de vocabulaire. À la différence de `authors`, il n'y a pas ici de défaut technique à corriger.

6.3 Points déjà réglés — à retirer de ta liste

- **Format de la date** : le schéma impose déjà un motif `YYYY-MM-DD`, plus un contrôle de date calendaire réelle dans `validate.py`.
- **`anyOf` → `oneOf`** : avec `additionalProperties: false`, une seule branche peut correspondre ; le résultat de validation est identique. C'est un gain de lisibilité des messages d'erreur, pas une correction de justesse.
- **`data_range` → notation tableau** : fait, sous la forme `col / rows`.
- **`listdataset.data` → `values`** : fait.
- **Homogénéisation des `requis`, `path` / `data`** : fait.

6.4 Restent à arbitrer

- `file_type` → `mime_type` : **abandonné** (décision JC). `file_type` reste.
- Préfixes ou motifs d'identifiants (`stg-`, `src-`, `set-`) contre les collisions entre espaces de noms plats.
- Flag données brutes / données d'analyse.
- LSA avec `fullfieldmeasurement` ou non.
- Maillage de l'éprouvette : `settings/specimen` ou `GenericDataSet` ?
- Un `GenericDataSet` peut-il exister sans source ?

7. Ce qui est plus riche en 2.x qu'en `apijc`

Pour équilibrer le tableau :

- **20 helpers guidés** (`add_camera_source`, `add_specimen_setting`, ...) sur `R3XAFfile` — que ta branche avait justement supprimés de `core.py`.
- **Trois moteurs de graphe** (`graphviz`, `pyvis` interactif, `networkx/matplotlib`) contre un seul.
- `Registry / RegistryItem`, et le dépôt `R3XA_REGISTRY` avec sa propre CI de validation.
- Introspection au-delà de la demande : `optional_fields()`, `required_fields()`, `missing_fields()`, `field_descriptions()`.
- Un stub de typage `core.pyi` généré, pour les complétions d'IDE.
- Une CI qui vérifie que le schéma embarqué ne dérive pas de `R3XA_SPEC`, et que `models.py` reste synchronisé avec le schéma.

8. Décisions prises (J-C. Passieux, 16 septembre)

#	Question	Réponse	Suite donnée
1	authors en {name, affiliation, orcid}	Oui	À implémenter avant la 2.0.0 finale — changement cassant, multi-dépôts
2	file_type → mime_type	Abandon	Rien à faire

#	Question	Réponse	Suite donnée
3	Palette : bordure colorée / fond clair	Validée	Rien à faire
4	Porter schemaplot	Non , « le rendu actuel est ok »	Rien à faire
5	Porter ses fichiers de travail	Non , réécriture par l'équipe	apijc peut être supprimée (tag apijc-snapshot)
6	Alias de classes	Oui si iso-propriétés	Condition vérifiée, alias livrés — voir 5.1
7	__repr__ des items	« Ce qui est le mieux »	Forme compacte conservée : un document.data_sources de 9 éléments resterait illisible autrement. print(item) donne le listing complet.
8	Points de schéma restants	Ok	△ À préciser — voir ci-dessous

Deux précisions encore nécessaires

Sur le point 8. Les items restants de la section 6.4 ne sont pas des propositions mais des *questions ouvertes* : préfixes d'identifiants (lesquels ?), flag données brutes / analyse (quel nom, quelles valeurs ?), LSA fullfieldmeasurement, emplacement du maillage d'éprouvette, GenericDataSet sans source. Un « Ok » global ne suffit pas à les implémenter : chacun demande un choix de conception. À reprendre point par point.

Sur le point 1. L'accord porte sur le principe. Restent à fixer avant écriture :

1. Noms des champs : name, affiliation, orcid — le schéma est en anglais.
2. Quels champs sont requis ? Proposition : name seul, les deux autres optionnels.
3. author_orcids : suppression pure, ou conservation en obsolète ? Proposition : suppression, le changement est cassant de toute façon.
4. Numéro de version du schéma : 2026.9.8 → 2026.9.16 ?
5. authors garde-t-il minItems : 1 ?

Le chantier touche **48 fichiers** dans R3XA_API (schéma embarqué, modèles générés, validate.py, 10 profils d'interface web, documentation, exemples, MATLAB, tests, CI), plus 7 dans R3XA_SPEC. R3XA_REGISTRY n'est pas concerné : ses items ne portent pas d'en-tête de document.